

Comparative Analysis of ML-Based Timestamp Manipulation Detection on NTFS Using \$LogFile and \$UsnJrnl Forensic Artifacts

Janna Victoria P. Cofreros
College of Computing and Information
Technologies
National University, Philippines
Manila, Philippines
cofrerosjp@students.national-u.edu.ph

Khyle R. Hernandez
College of Computing and Information
Technologies
National University, Philippines
Manila, Philippines
hernandezkr@students.national-u.edu.ph

Sonya G. Lee
College of Computing and Information
Technologies
National University, Philippines
Manila, Philippines
leesg@students.national-u.edu.ph

Jaira M. Marcial
College of Computing and Information
Technologies
National University, Philippines
Manila, Philippines
marcialjm1@students.national-u.edu.ph

Josephine Andrea B. Padilla
College of Computing and Information
Technologies
National University, Philippines
Manila, Philippines
padillajb1@students.national-u.edu.ph

Rogel M. Labanan
College of Computing and Information
Technologies
National University, Philippines
Manila, Philippines
rmlabanan@national-u.edu.ph

Abstract— Timestamp manipulation, or timestomping, is a critical anti-forensic technique used by cybercriminals to obscure the timeline of malicious activity by altering file metadata, specifically the modified, accessed, changed, and created or birth timestamps in the New Technology File System. Existing rule-based detection tools often struggle to differentiate between malicious timestomping and benign system activities, leading to high false positives that compromise the efficiency of forensic investigation. This paper addresses this gap by reframing timestomping detection as an anomaly detection problem with a severe class imbalance. An artificial intelligence-driven methodology was proposed and evaluated by using and comparing the supervised Random Forest classifier and the unsupervised Isolation Forest model for anomaly detection, trained on a controlled, labeled dataset derived from the feature-engineered correlation of the New Technology File System’s journaling artifacts, the Log File and the Update Sequence Number Change Journal. The final model classifies file system events with a measurable confidence score. Between the two models evaluated, the Random Forest classifier has demonstrated superior performance, significantly reducing the false positive rate compared to the Isolation Forest model. This validation of a robust data-driven approach for timestomping detection, which is also packaged as a Python Data Source Ingest Module plugin prototype for Autopsy that automates the entire detection pipeline, provides digital forensic investigators with a confidence-scored result to identify file manipulation, thereby enhancing the reliability and efficiency of artifact anomaly detection in the digital forensic workflow.

Keywords— Timestamp manipulation, New Technology File System (NTFS) artifacts analysis, Digital Forensics, anomaly detection, Machine Learning.

I. INTRODUCTION

Cybercrime continues to escalate globally, exploiting vulnerabilities in digital systems to conceal malicious activity. One of the most common strategies employed by cybercriminals is anti-forensics, which involves deliberately manipulating digital evidence to hinder investigations [1]. Moreover, they are

increasingly utilizing timestamp manipulation, also known as “timestomping,” to change the date and time information associated with files, so it creates illusion of no suspicious activity. This activity causes challenges for digital forensic investigators because they need to determine the timeline of events, including when the files are created, accessed, modified, or deleted. Timestomping creates a major threat that weakens the strength of forensic evidence and integrity of the investigation. Given the growing reliance on digital evidence in criminal cases and organizational investigations, accurate timeline reconstruction has become central to modern digital forensics.

Timeline reconstruction primarily relies on system artifacts, such as logs, registry data, and file system metadata, the descriptive information stored about files. In Windows, the New Technology File System (NTFS) stores multiple timestamp attributes—Modified, Accessed, Changed, and Birth (MACB)—which reflects how files have been created, used, or altered [2]. Discrepancies observed in these attributes can reveal if tampering occurred and can help in reconstructing sequence of file system events that occurred [3].

The NTFS transaction log, which is the \$LogFile is frequently used for its capabilities to record file system transactional changes, including file creation, deletion, and data change. Complementing this, the \$UsnJrnl or Update Sequence Number Journal, acts as the supporting artifact because it keeps a log of all file activities, such as file creation, deletion, renaming, and modification [4], [2]. This NTFS journal artifact provides the historical record of file changes and shows what change occurred. These NTFS artifacts are important in identifying user actions, linking digital behavior to specific timeframes, and validating other sources of evidence.

Forensic studies show that both legitimate user operations and malicious timestomping tools can generate irregular and inconsistent timestamp patterns, making it difficult to distinguish between benign and malicious behavior [5]. As a

result, false positives remain a persistent challenge in timestamp analysis. Effective detection of anti-forensic activity like timestamping cannot be done by looking at a single artifact. Doing so may lead to false positives and incomplete findings. Checking multiple NTFS artifacts improves timestamp manipulation detection accuracy and strengthens the credibility of forensic conclusions. However, the volume of records in modern investigations often exceeding hundreds of thousands of entries, making manual analysis time-consuming and impractical. To address this issue, this study proposes a data-driven methodology comparing several algorithms such as: Random Forest, XGBoost, LightGBM, and Logistic Regression in detecting timestamp manipulation on NTFS using \$LogFile and \$UsnJrnl artifacts. By analyzing metadata in both NTFS artifacts, the comparative analysis highlights differences in timestamping detection accuracy and false positives rates. This provides insight into selecting the most effective algorithm in detecting timestamp manipulations within Windows NTFS. With automated detection, AI-driven models will be able to ensure greater efficiency and higher accuracy in timeline reconstruction.

Ultimately, the primary objective of this study is to integrate digital forensic reasoning with machine learning to create a Python Data Source Ingest Module for Autopsy, a widely used digital forensics platform. This plugin will aid investigators in automating timestamp analysis and providing results with confidence scores and anomaly detection results.

A. Significance of the Study

The study contributes to cybersecurity and digital forensics by integrating machine learning in NTFS artifact analysis, specifically \$LogFile and \$UsnJrnl, in automating timestamp detection. This study bridges traditional rule-based methods, and modern AI-driven approaches, enabling more data-driven and adaptive forensic analysis.

The results benefit law enforcement agencies (PNP-ACG) and digital forensic practitioners by improving timestamp detection accuracy and reliability of digital evidence. Furthermore, the study will provide the academic and research community with a model and framework that may serve as their foundation for future AI-driven forensic tools.

B. Objectives of the Study

The general objective of this study is to develop, evaluate, and optimize a machine learning framework for timestamping detection in NTFS artifacts, specifically the \$LogFile and \$UsnJrnl, through comparative analysis and hyperparameter tuning in preparation for deployment as a Python Data Source Ingest Module plugin prototype for Autopsy.

The specific objectives of the study are the following: (1) to compare the performance of four different trained machine learning classification models for detecting timestamping within a feature-engineered dataset derived from correlated \$LogFile and \$UsnJrnl metadata, (2) to refine and validate

the superior machine learning model to reliably classify malicious and benign file system events with a measurable confidence score, and (3) to optimize the best-performing model through hyperparameter tuning in preparation for integration as a Python Data Source Ingest Module plugin prototype for Autopsy that automates the timestamping detection pipeline as a triage tool for digital forensic investigators in their forensic workflows.

II. REVIEW OF RELATED STUDIES

A. Synthesis of Related Studies

The reviewed studies collectively emphasize the increasing complexity of detecting timestamp manipulation within NTFS. Study [2] shows that inconsistencies across \$MFT, \$LogFile, and \$UsnJrnl can indicate timestamping, yet rule-based approaches still produce high false positives due to benign system activities that resemble manipulation. Work in [5] further demonstrates how NTFS timestamps behave differently across operating system versions and software workflows, making isolated timestamp analysis unreliable.

Research in [6] expands this by showing that NTFS timestamps can correspond to multiple valid file histories, highlighting the need for multi-artifact correlation instead of direct linear interpretation. Broader anti-forensic literature also underscores the sophistication of obfuscation methods. Reference [7] classifies timestamping within wider anti-forensic strategies that distort or conceal evidence, while [8] shows how timestamp tampering undermines event reconstruction and necessitates more advanced analytical models.

Building upon these foundations, a recent study [2] made significant advances by developing forensic detection algorithms specifically designed for \$LogFile and \$UsnJrnl artifacts. Their work addressed a critical limitation in prior research: the prevalence of false positives caused by legitimate file system tunneling behavior in Windows NTFS. By analyzing actual APT malware samples, the study demonstrated that cross-artifact validation between \$LogFile transaction records and \$UsnJrnl change journal entries provides a robust detection mechanism for distinguishing malicious timestamp manipulation from benign system operations. Their proposed algorithms extract temporal deltas between \$STANDARD_INFORMATION (\$SI) and \$FILE_NAME (\$FN) timestamps, correlate reason codes across journal artifacts, and compute anomaly scores based on multi-attribute discrepancies. These techniques form the methodological foundation for the feature engineering approach employed in the current study, particularly the use of location-agnostic features to ensure generalizability across diverse malware families and system configurations.

Recent studies on evasion techniques [9] and data trace reliability [10] further illustrate how attackers exploit metadata ambiguity and how forensic tools must adapt to maintain evidentiary integrity. Supporting this, [11] highlights the forensic relevance of timestamp-related contact artifacts,

whereas [12] identifies the increasing importance of AI-driven methodologies in modern forensic workflows.

Overall, the literature identifies three key gaps: (1) NTFS timestamps are inherently ambiguous and sensitive to benign variation, (2) modern anti-forensic techniques reduce the reliability of traditional rule-based detection, and (3) machine-learning-based, multi-artifact analysis remains underexplored. These gaps justify developing a data-driven, AI-enabled approach using correlated \$LogFile and \$UsnJrnl artifacts for more reliable detection of timestomping.

III. METHODOLOGY

The methodological framework for this study is a comparative data-driven approach designed to address the severe class imbalance inherent in timestomping detection. It is divided into four phases: data preparation, feature engineering, the design of the comparative machine learning pipeline, and optimization of the best performing model from the comparative analysis.

A. Data Acquisition & Preparation

The foundation of the research is a forensically relevant, labeled dataset constructed from 18 distinct forensic cases provided by Junghoon Oh: 12 Performance Evaluation (PE) cases and 6 APT malware datasets (APT17, APT19, APT28, APT29, DarkHotel663, DarkHotelb663). This diverse dataset spans synthetic timestamp manipulation scenarios and real-world Advance Persistent Threat campaigns, ensuring model generalizability across attack patterns and file system behaviors.

The data preparation workflow began with Parsing and Extraction, where raw \$LogFile and \$UsnJrnl artifacts were processed using the NTFS Log Tracker tool to convert NTFS structures into structured CSV records. This conversion preserved critical forensic metadata, including LSN values, redo and undo operation details, VCN targets, and \$STANDARD_INFORMATION timestamps from \$LogFile, as well as USN entries, reason codes, file attributes, and change-journal metadata from \$UsnJrnl.

After extraction, the dataset underwent Filtering and Labeling guided by detection patterns described in [2]. Timestamp manipulation indications such as Time Reversal Events, and Update Resident Value operations were isolated from \$LogFile, while the BASIC_INFO_CHANGE sequence, particularly one followed with File_Closed pattern, captured all labeled timestomped activity in \$UsnJrnl. Data labeling was executed by cross-referencing records with the tool's Suspicious-Behavior-Detected output, consistently assigning the anomaly class to timestomped events.

A critical step was Cross-Artifact Integration, where the individual sets of \$LogFile and \$UsnJrnl records were consolidated into unified case files. Instead of a simple outer join, a smart-union approach was applied to merge \$LogFile and \$UsnJrnl entries that referenced the same file within the $\pm$

window, enabling cross-artifact validation while preserving single-source evidence. This method captured cross-correlated events as well as LogFile-only and UsnJrnl-only traces. To mitigate false positives caused by legitimate Windows behavior, the pipeline included a tunneling-detection stage based on [2], flagging instances where Windows reuses cached \$STANDARD_INFORMATION timestamps for files deleted or renamed within a 15-second window.

Finally, the unified case outputs were created by vertically concatenating all unified case files into a single chronologically ordered dataset. This aggregated dataset is designated for a case-based stratified split to prevent data leakage and ensure the model is robustly tested on entirely unseen forensic scenarios, simulating real-world application. Overall, the dataset handling produced 280 confirmed timestomped events with over 280,000 benign events, revealing a pronounced class imbalance of approximately 1:1010.

B. Feature Engineering

Following the data collection and preprocessing phase, an extensive feature engineering process was conducted to transform the raw forensic event data into a machine learning-ready format. The main goal was to develop features capable of detecting the nuanced patterns and characteristics traces left by timestamp manipulation tools. The developed features were logically grouped into six core categories:

1) *File Type and Characteristic Indicators.* These features capture file classification patterns and metadata properties, including binary flags, for executable files, system files, hidden files, compressed archives, and files with suspicious extensions (e.g., .scr, .pif, and .bat), as well as filename length and path depth metrics. This category addresses the observation that timestamp manipulation frequently targets specific file types, particularly executables and system binaries, and that attackers often use short, inconspicuous filenames or deeply nested directory to evade detection. The path depth feature measures directory nesting levels without flagging specific locations, ensuring location-agnostic analysis.

2) *Temporal Clustering and Delta Features.* These features were created to capture time-based behavioral patterns that deviate from normal system activity or suggest coordinated manipulation attempts. This includes analyzing event density within temporal windows, measuring time elapsed since the previous event on the same file, calculating event frequency per file and per case, and deriving timestamp deltas between \$FILE_NAME (\$FN) and \$STANDARD_INFORMATION (\$SI) attributes. The rationale is that timestomping tools often generate clustered bursts of BASIC_INFO_CHANGE events within short timeframes and create discrepancies between \$MFT \$FN and \$SI timestamps, patterns that are rare in legitimate file operations.

3) *\$LogFile-Derived Anomaly Flags*. These features capture logical inconsistencies and manipulation indicators extracted from \$LogFile Time Reversal events and parsed detail fields. Flags include whether timestamps were copied from another file, whether specific timestamp types were changed to past, and whether nanosecond precision was set to zero, a characteristic artifact of certain timestomping tools that manipulate timestamps at second-level granularity. Such anomalies violate fundamental filesystem semantics and serve as strong indicators of manipulation.

4) *\$UsnJrnl Pattern Features*. These features encode the forensic event sequences and patterns found in the \$UsnJrnl change journal, analyzing the presence of specific reason codes and their combinations. Flags include indicating metadata-only modifications, file handle closure, file attribute modifications, and detecting the characteristic BASIC_INFO_CHANGE + FILE_CLOSED sequence without accompanying data operations. This is relevant because timestomping tools generate distinctive event sequences, particularly isolated BASIC_INFO_CHANGE followed immediately by FILE_CLOSED, that differ from the multi-step reason code patterns produced by normal application workflows.

5) *Cross-Artifact Validation Features*. As a key innovation of this study, this category comprises features derived from correlated \$LogFile and \$UsnJrnl data, emphasizing the complementary nature of these two NTFS journals. Features include binary presence flags indicating whether timestomping indicators appear in each artifact, and a cross_artifact_validation_score that quantifies the strength of cross-artifact corroboration. This leverages the principle that legitimate file system operations maintain temporal coherence across artifacts, whereas timestamp manipulation often introduces subtle inconsistencies detectable through cross-validation. Events flagged by both journals receive higher suspicion scores than single-artifact detections

6) *Composite Forensic Scoring Features*. This category aggregates multiple anomaly indicators into weighted risk scores. The timestamp_manipulation_pattern_score combines temporal anomalies, delta discrepancies, and \$LogFile flags into a single composite metric, while the file_system_tunneling_confidence score distinguishes legitimate Windows file system tunneling behavior from malicious manipulation by analyzing the temporal proximity of file deletion/rename events preceding BASIC_INFO_CHANGE operations. These composite features provide high-level abstractions that synthesize lower-level forensic signals into interpretable risk metrics suitable for investigator review and model decision-making.

C. Machine Learning Pipeline Design

The machine learning pipeline was designed to compare four supervised classification algorithms: Random Forest,

XGBoost, LightGBM, and Logistic Regression, in their ability to detect timestamp manipulation based on features engineered from the \$LogFile and \$UsnJrnl artifacts. Since existing literature does not provide a directly comparable baseline that integrates both artifacts in a unified model, this research establishes an initial benchmark for evaluating supervised learning in NTFS-based timestomping detection.

Each model was trained using the same forensic dataset and the same case-aware splitting strategy to ensure methodological consistency. Entire cases were assigned exclusively to either the training or test set to prevent cross-case leakage and to simulate realistic deployment conditions in which a model must generalize to completely unseen investigations. Given the extreme class imbalance in the dataset, class imbalance handling strategies were applied during training. For Random Forest and Logistic Regression, class weighting was used to increase the penalty for misclassifying minority-class timestomped events. For XGBoost and LightGBM, the scale_pos_weight parameter was set according to the observed class ratio to emphasize the minority class. SMOTE was evaluated in preliminary experiments but ultimately excluded from final training because it introduced artificial variation and increased the risk of overfitting.

Random Forest served as the primary non-linear baseline due to its robustness to noise, ability to model complex feature interactions, and interpretability through feature importance scores. XGBoost and LightGBM were included to evaluate the performance of gradient-boosted tree methods, which efficiently handle large feature sets and class imbalance while capturing subtle forensic patterns. Logistic Regression was included as a linear baseline to determine how well timestamp manipulation can be detected through simple, interpretable relationships between features without hierarchical decision structures or boosting.

D. Model Evaluation Framework

Models were evaluated using metrics suited for highly imbalanced forensic datasets. F1-score was selected as the primary performance metric, balancing precision and recall. Recall was prioritized to ensure timestomped events were detected, while precision was monitored to avoid overwhelming analysts with false positives. Additional metrics including false positive rate, and false negative rate provided operational insight into real-world performance.

Comparative analysis across the four algorithms was conducted on identical test sets. Random Forest feature importance rankings identified the most predictive forensic indicators, while XGBoost and LightGBM were assessed for predictive strength and stability across diverse cases. Logistic Regression coefficients provided a transparent view of the features with the strongest linear influence on timestomping. This multi-model evaluation framework establishes a baseline for machine learning-based timestamp manipulation detection and informs future research on forensic model optimization.

E. Prototype Integration

The prototype will be developed as a Python Data Source Ingest Module for the Autopsy digital forensics platform. The workflow will begin within the Autopsy interface, where it will allow an investigator to add a data source, such as a disk image, to their created case. This component will then automatically identify and export the \$LogFile and \$UsnJrnl raw files to an Autopsy case-specific folder.

Once these artifacts are exported, the core of the timestamp detection framework, an external Python 3 processing with a trained machine learning model, will be automatically invoked. This external process will serve as the analytical core of the prototype, first parsing the raw NTFS artifacts into a structured format. After this, it will merge the data into a master timeline. With the Random Forest model embedded, the engineered feature set for each file system event will be passed to the external processing layer. Finally, this engineered data will be run through the trained model to generate the final predictions.

The last process will give these results back to the investigator. It will read the output file generated by the external process layer and will display the findings, including the confidence score for each event, within Autopsy's results viewer interface. This final step will provide a clear and actionable report, allowing the investigator to quickly identify and prioritize the most suspicious file system activities.

IV. PRELIMINARY RESULTS & DISCUSSION

This section presents a quantitative comparison of the four supervised machine learning models: Random Forest, XGBoost, LightGBM, and Logistic Regression, evaluating their performance in detecting timestamp manipulation.

A. Preliminary Results

1) *Model Performance Comparison.* All models trained and tested on the same dataset, representing a severe class imbalance ratio. The performance on the held-out test set is summarized in the Table 1 below.

TABLE I. COMPARATIVE MODEL PERFORMANCE METRICS

<i>Metric</i>	<i>Random Forest</i>	<i>XGBoost</i>	<i>LightGBM</i>	<i>Logistic Regression</i>
F1-Score	0.6000	0.1720	0.0120	0.0538
Recall	0.9231	0.1026	0.0769	0.9615
Precision	0.4444	0.5333	0.0065	0.0277
False Positive Rate % (FPR)	0.20	0.02	2.04	5.86
False Negative Rate % (FNR)	7.69	89.74	92.31	3.85
Detected	72/78	8/78	6/78	75/78
False Alarms	90	7	915	2,633

2) *Analysis of Baseline Results.* Looking at the results, Random Forest achieved the highest overall performance among the models, demonstrating a strong detection rate of

92.31% by correctly identifying 72 of 78 timestamped events. Its precision was moderate at 44.44%, resulting in 90 false alarms among 162 total alerts, and the F1-score of 0.6000 indicated the best balance between precision and recall across all evaluated algorithms.

By contrast, XGBoost and LightGBM exhibited severe underfitting despite the application of class imbalance handling strategies through the `scale_pos_weight` parameter set to 1177.75. Both models missed more than 89% of timestamped events, with recall values ranging from 7.69% to 10.26%, rendering them unsuitable for forensic detection in this scenario. This performance is likely due to gradient boosting's conservative decision boundaries, which struggle to capture rare events under extreme class imbalance. Logistic Regression displayed very high recall (96.15%), correctly identifying 75 of 78 timestamped events, but its precision was unacceptably low at 2.77%, generating 2,633 false alarms, or approximately 35 false positives for every true detection. The linear decision boundary inherent to Logistic Regression appears insufficient to model the complex interactions present in timestamp manipulation patterns.

Overall, Random Forest emerged as the most effective baseline supervised model, achieving a favorable trade-off between detection completeness and analyst workload, and providing a robust foundation for subsequent model optimization and forensic deployment.

3) *Hyperparameter Optimization and Final Model Selection.* Because Random Forest performed best in the baseline evaluation, its next phase focused on tuning its settings to improve detection accuracy. A systematic search was performed using GridSearchCV with five-fold case-aware cross-validation, testing 324 different combinations of key parameters. These included the number of trees in the forest, the maximum depth of each tree, the minimum number of samples required to split a node or be at a leaf, and how many features to consider when making a split. Class weighting was set to balanced to address the extreme imbalance between normal and timestamped events. The search identified the best configuration as 100 trees, a maximum depth of 10, at least 20 samples to split a node, 2 samples per leaf, and using the square root of all features for each split. This optimized setup improves the model's ability to detect timestamped events while reducing false alarms, ensuring reliable performance on unseen forensic cases. Table 2 presents the comparison of the baseline and optimized Random Forest Performance.

TABLE II. BASELINE VS OPTIMIZED RANDOM FOREST PERFORMANCE

<i>Metric</i>	<i>Baseline</i>	<i>Optimized (Phase 5)</i>	<i>Difference</i>
F1-Score	0.6000	0.8820	+0.2820
Recall	92.31%	91.03%	-1.28%
Precision	44.44%	85.54%	+41.10%
FPR	0.20%	0.027%	-0.173%

Metric	Baseline	Optimized (Phase 5)	Difference
FNR	7.69%	8.97%	+1.28%
False Alarms	90	12	-78
Detected Attacks	72/78	71/78	-1

4) *Feature Importance Analysis.* The Random Forest model provided crucial interpretability via feature importance analysis, identifying the forensic indicators most predictive of manipulation. The top features reveal that the model relies heavily on pattern recognition within the journals rather than simple timestamp checks.

TABLE III. TOP TEN RANDOM FOREST FEATURE IMPORTANCE

Rank	Feature	Importance	Category
1	usn_usn	19.18	\$UsnJrnl Pattern Features
2	usn_event_info	11.20	\$UsnJrnl Pattern Features
3	path_depth	9.73	File Characteristics
4	events_in_5min_window	7.36	Temporal Clustering
5	events_in_1min_window	6.86	Temporal Clustering
6	timestamp_manipulation_pattern_score	6.58	Composite Forensic Scoring
7	filename_length	4.88	File Characteristics
8	cross_artifact_validation_score	4.68	Cross-Artifact
9	usn_complete_manipulation_pattern	4.33	\$UsnJrnl Pattern Features
10	is_archive	4.28	File Characteristics

B. Discussion

1) *Random Forest Superiority.* The optimized Random Forest model revealed a multi-layered detection strategy for timestamp manipulation, with feature importance reflecting the relative contribution of different forensic signals. The model relies primarily on \$UsnJrnl pattern features, which constitute 34.71% of total importance. The top contributors within this category include the Update Sequence Number (usn_usn), event reason codes (usn_event_info), and the complete manipulation pattern indicator (usn_complete_manipulation_pattern). These features provide temporal ordering, distinguish isolated timestamp modifications from legitimate operations, and capture coordinated event sequences, validating the dominance of UsnJrnl artifacts in detecting timestomping.

Temporal clustering features account for 14.22% of importance, capturing coordinated attack behaviors.

Specifically, modifications occurring within one- and five-minute windows highlight bursts of automated timestomping activity, allowing the model to distinguish rapid, attacker-driven file changes from organic system operations. File characteristics contribute 18.89% of importance, including directory depth (path_depth), filename length, and archive file flags, supporting a location-agnostic design that generalizes across diverse file system structures and avoids dataset-specific overfitting.

Cross-artifact validation (cross_artifact_validation_score, 4.68%) and composite forensic scoring (timestamp_manipulation_pattern_score, 6.58%) provide additional corroboration by integrating evidence across \$LogFile and \$UsnJrnl and aggregating multiple anomaly signals. While these features are less influential individually, they play a critical role in reducing false positives and reinforcing detection confidence.

Overall, the model learned a hierarchical approach: first, primary signal detection through UsnJrnl patterns; second, behavioral context via temporal clustering; third, file-level characteristics; and fourth, corroboration across artifacts and composite anomaly scoring. This multi-layer strategy explains the model’s high performance metrics: a 91.03% recall by detecting all ground truth timestomped events, 85.54% precision by filtering benign high-frequency events, and a low false positive rate (0.027%). The feature importance analysis demonstrates that raw, granular forensic signals—particularly cross-artifact temporal patterns—drive detection, while location-agnostic features ensure robust generalization across synthetic and real-world APT datasets.

The profound difference in performance, particularly the sixty-four point seven percent higher Recall achieved by the Random Forest classifier, validates the hypothesis that supervised learning with explicit labeling is essential for timestamp manipulation detection. The findings suggest that the artifacts left by timestomping tools (like NTimeStomp and SetMACE) do not consistently present as extreme statistical outliers, which is the foundational assumption of anomaly detection methods like Isolation Forest. Instead, the manipulation creates subtle inconsistencies, such as cross-artifact mismatches and event sequence disruptions, that require contextual pattern learning.

The high importance of features like usn_event_encoded and the cross-artifact features demonstrates that the model is learning the relationship between events, a capability that unsupervised methods lack. Traditional timestamp anomaly flags contributed only marginally to the overall prediction, further underscoring the need for a non-linear, feature-rich classification approach.

2) *Forensic Triage Implications.* From the perspective of a digital forensics practitioner, the model comparison provides clear and distinct operational guidance.

The Random Forest classifier offers an acceptable compromise between detection and efficiency. It successfully identifies the majority of malicious files while simultaneously achieving a massive reduction in the scope of manual review. The resulting number of incorrectly flagged benign files is a manageable burden when weighed against the hundreds of thousands of files an investigator would otherwise be forced to review.

Logistic Regression, while achieving high recall, produces an unmanageable number of false alarms, and XGBoost and LightGBM severely underperform, missing the vast majority of manipulated files. These results indicate that ensemble-based tree models with careful hyperparameter tuning provide the most reliable and efficient approach for operational forensic triage, whereas linear and gradient-boosting approaches are either prone to excessive false positives or fail to detect timestomping events entirely.

C. Limitations

The current findings, while establishing a methodological benchmark, are subject to limitations that necessitate further research. The dataset used to train and validate the models is derived exclusively from controlled environments, with manipulations constrained to specific, known anti-forensic tools. Consequently, the model's generalization to other timestomping techniques, such as those employing Windows API calls or specialized PowerShell scripts, remains to be empirically tested. Future work must prioritize expanding the training dataset to include a wider variety of adversarial attack signatures.

While the Phase 5 optimized Random Forest classifier demonstrated clear superiority over alternative approaches, including baseline Logistic Regression, XGBoost, and LightGBM, further comparative analysis against other high-performance supervised algorithms is warranted. This would allow the identification of the absolute optimal model for operational deployment, ensuring both high detection rates and minimal analyst workload.

Additionally, although the Random Forest model achieved strong recall and precision, the results suggest that further refinement is possible. Future efforts may explore alternative class imbalance handling strategies, such as slightly more aggressive synthetic oversampling or adjusted class weighting penalties, to optimize the trade-off between recall and precision. The ultimate objective is to minimize missed timestomped files while maintaining a manageable triage burden. The final step remains the full integration of the optimal model into the Autopsy Python Ingest Module for operational validation and real-world deployment.

ACKNOWLEDGMENT

The authors gratefully acknowledge the Digital Forensics and Information Security Research Laboratory at the College of Computing and Information Technologies (CCIT) of the National University, Philippines for providing institutional and computational resources that were instrumental for the study's forensic data extraction, model training and testing, and collaborative manuscript writing. The authors also wish to thank their classmates for their valuable technical discussions and collaborative assistance, which significantly informed the study's initial model and prototype development, as well as the manuscript preparation.

REFERENCES

- [1] A. Rathbun, "Anti-forensic tactics: Timestomping," *Kroll*, 2022. [Online]. Available: <https://www.kroll.com/en/publications/cyber/anti-forensic-tactics/anti-forensics-tactics-timestomping>
- [2] J. Oh, S. Lee, and H. Hwang, "Forensic Detection of Timestamp Manipulation for Digital Forensic Investigation," *IEEE Access*, vol. 12, pp. 72544–72565, 2024. doi: 10.1109/ACCESS.2024.3395644.
- [3] A. Thakur, "Mastering NTFS forensics: A comprehensive guide for practitioners," *Medium*, Aug. 13, 2025. [Online]. Available: <https://medium.com/@ashutoshthakurofficial/mastering-ntfs-forensics-a-comprehensive-guide-for-practitioners-1236618fd49>.
- [4] J. Oh, S. Lee, and H. Hwang, "NTFS Data Tracker: Tracking file data history based on \$LogFile", *Forensic Science International: Digital Investigation*, vol. 39, p. 301309, Dec. 2021, doi: 10.1016/j.fsidi.2021.301309.
- [5] M. Galhuber and R. Luh, "Time for Truth: Forensic Analysis of NTFS Timestamps," in *Proc. 16th Int. Conf. Availability, Reliability and Security*, Vienna, Austria: ACM, Aug. 2021, pp. 1–10. doi: 10.1145/3465481.3470016.
- [6] J. Bouma, H. Jonker, V. Van Der Meer, and E. Van Den Aker, "Reconstructing Timelines: From NTFS Timestamps to File Histories," in *Proc. 18th Int. Conf. Availability, Reliability and Security*, Benevento, Italy: ACM, Aug. 2023, pp. 1–9, doi: 10.1145/3600160.3605027.
- [7] Z. Abdullahi, N. Sagarwal, and M. Soni, "An Overview of Anti-forensic Techniques and their Impact on Digital Forensic Analysis," Feb. 2023.
- [8] C. Vanini, J. Gruber, C. Hargreaves, Z. Benenson, F. Freiling, and F. Breiting, "Strategies and Challenges of Timestamp Tampering for Improved Digital Forensic Event Reconstruction (extended version)," arXiv:2501.00175, Dec. 2024. doi: 10.48550/arXiv.2501.00175.
- [9] L. Zhuo, S. Luo, S. Tan, H. Chen, B. Li, and J. Huang, "Evading Detection Actively: Toward Anti-Forensics against Forgery Localization," arXiv, 2023, doi: 10.48550/ARXIV.2310.10036.
- [10] Z. Cao, B. Gao, Z. Liu, X. Xiong, B. Wang, and C. Pei, "Data trace as the scientific foundation for trusted metrological data: a review for future metrology direction," *PeerJ Comput. Sci.*, vol. 11, p. e3106, Aug. 2025, doi: 10.7717/peerj-cs.3106.
- [11] D.-Y. Kao, "Forensic Exchange Analysis of Contact Artifacts on Data Hiding Timestamps," *Applied Sciences*, vol. 10, no. 13, p. 4686, Jul. 2020, doi: 10.3390/app10134686.
- [12] D. Dunsin, M. C. Ghanem, K. Ouazzane, and V. Vassilev, "A comprehensive analysis of the role of artificial intelligence and machine learning in modern digital forensics and incident response," *Forensic Science International: Digital Investigation*, vol. 48, p. 301675, Mar. 2024, doi: 10.1016/j.fsidi.2023

